NAME: K. SWARNA VARSHINI

REG.NO: 192324081

COURSE CODE: CSA0920

GITHUB LINK:

https://github.com/kswarna1/CSA0920-JAVA.git

Q1– GUI DESIGN REVIEW

Aim:

To analyse and redesign a student fee-payment GUI application using Java Swing by identifying event-handling problems, separating interface and payment logic, implementing event listeners, validating user input and displaying payment results.

Theory:

Java Swing is used to develop graphical user interfaces (GUI). It provides components such as JFrame, JLabel, JTextField and JButton.

The Event Delegation Model is used to handle user actions. When a user clicks a button, an event is generated and handled by a registered listener.

The main concepts used are:

1. **Separation of concerns:** The GUI and business logic are maintained in separate classes.
2. **Event handling:** ActionListener handles button-click events.
3. **Input validation:** Checks whether the registration number and payment amount are valid.
4. **SwingWorker:** Performs background operations without freezing the GUI.
5. **JOptionPane:** Displays success and error messages.
6. **WindowAdapter:** Handles window-closing events.
7. **PreparedStatement:** Helps prevent SQL injection when database operations are implemented.

The redesigned application contains two main classes:

- FeeWindow – collects input and displays results.
- FeeService – validates input and processes the payment.

Java Program:

File 1: FeeService.java

```
import java.math.BigDecimal;

public class FeeService {

    public String pay(String regNo, String amountText) {

        if (regNo == null || regNo.trim().isEmpty()) {

            throw new IllegalArgumentException(

                "Enter registration number");

        }

        BigDecimal amount;

        try {

            amount = new BigDecimal(amountText.trim());

        } catch (Exception e) {

            throw new IllegalArgumentException(

                "Enter a valid amount");

        }

        if (amount.compareTo(BigDecimal.ZERO) <= 0) {

            throw new IllegalArgumentException(

                "Amount must be positive");

        }

        return "Payment Successful!\n"

                + "Registration Number: " + regNo.trim()

                + "\nAmount Paid: Rs. " + amount.setScale(2);

    }

}
```

File 2: FeeWindow.java

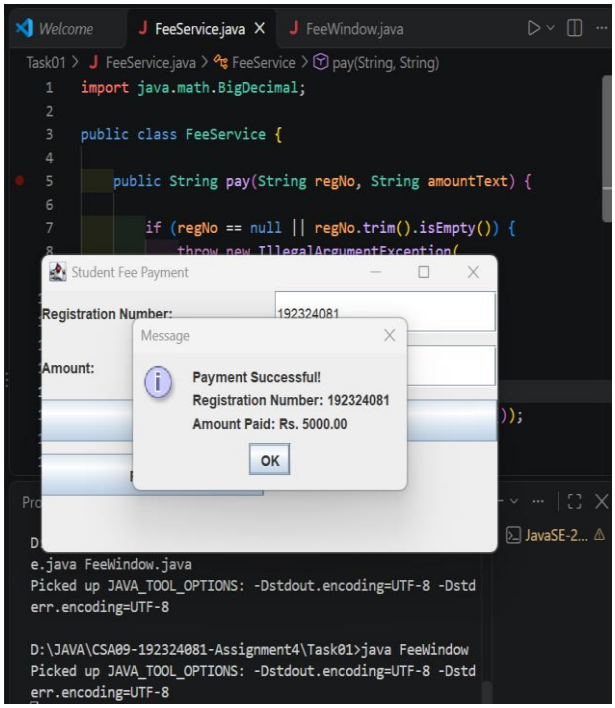
```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class FeeWindow extends JFrame {
    JTextField regNo = new JTextField(15);
    JTextField amount = new JTextField(15);
    JButton pay = new JButton("Pay");
    JButton clear = new JButton("Clear");
    JButton receipt = new JButton("Receipt");
    JLabel status = new JLabel("Enter payment details");
    FeeService service = new FeeService();
    String lastReceipt = "";
   boolean processing = false;
    public FeeWindow() {
        setTitle("Student Fee Payment");
        setLayout(new GridLayout(5, 2, 10, 10));
        add(new JLabel("Registration Number:"));
        add(regNo);
        add(new JLabel("Amount:"));
        add(amount);
        add(pay);
        add(clear);
        add(receipt);
        add(status);
```

```
pay.addActionListener(e -> makePayment());
clear.addActionListener(e -> {
    if (!processing) {
        regNo.setText("");
        amount.setText("");
        status.setText("Fields cleared");
        lastReceipt = "";
    }
});
receipt.addActionListener(e -> {
    if (lastReceipt.isEmpty()) {
        JOptionPane.showMessageDialog(this,
                "No receipt available");
    } else {
        JOptionPane.showMessageDialog(this, lastReceipt);
    }
});
addWindowListener(new WindowAdapter() {
    @Override
    public void windowClosing(WindowEvent e) {
        if (processing) {
            JOptionPane.showMessageDialog(
                    FeeWindow.this,
                    "Payment is in progress. Please wait.");
```

```
            } else {
                dispose();
            }
        }
    });
    setSize(450, 250);
    setLocationRelativeTo(null);
    setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
}
private void makePayment() {
    if (processing) {
        return;
    }
    processing = true;
    pay.setEnabled(false);
    status.setText("Processing payment...");
    SwingWorker<String, Void> worker =
            new SwingWorker<String, Void>() {
        @Override
        protected String doInBackground() {
            return service.pay(
                    regNo.getText(),
                    amount.getText());
        }
```

```
    @Override
    protected void done() {
        processing = false;
        pay.setEnabled(true);
        try {
            lastReceipt = get();
            status.setText("Payment Successful");
            JOptionPane.showMessageDialog(
                    FeeWindow.this, lastReceipt);
        } catch (Exception ex) {
            lastReceipt = "";
            status.setText("Payment Failed");
            JOptionPane.showMessageDialog(
                    FeeWindow.this,
                    ex.getCause() != null
                            ? ex.getCause().getMessage()
                            : ex.getMessage(),
                    "Error",
                    JOptionPane.ERROR_MESSAGE);
        }
    }
};
```

```
        worker.execute();
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() ->
                new FeeWindow().setVisible(true));
    }
}
```



Q2– FROM REQUIREMENT TO EVENT-DRIVEN DESIGN

Aim

To design and develop a Java Swing-based Hospital Outpatient Appointment Desk that allows patients to enter their Patient ID, select a department, choose a doctor and time slot, and book an appointment using event handling.

Theory

The Hospital Outpatient Appointment Desk is an event-driven GUI application developed using Java Swing.

- **JFrame:** Creates the main application window.
- **JLabel:** Displays field names.
- **JTextField:** Accepts the Patient ID.
- **JComboBox:** Allows users to select departments, doctors and time slots.
- **JButton:** Performs appointment booking.
- **ActionListener:** Handles button clicks and department selection.
- **WindowAdapter:** Handles window-closing events.
- **SwingWorker:** Performs the booking operation in the background so the GUI remains responsive.

Java Program

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class Task02 extends JFrame {
    JTextField patientId;
    JComboBox<String> department, doctor, timeSlot;
    JButton book;
    JLabel status;
    boolean bookingInProgress = false;
    SwingWorker<String, Void> worker;
    public Task02() {
        setTitle("Hospital Appointment Desk");
        setLayout(new GridLayout(6, 2, 10, 10));
        patientId = new JTextField();
```

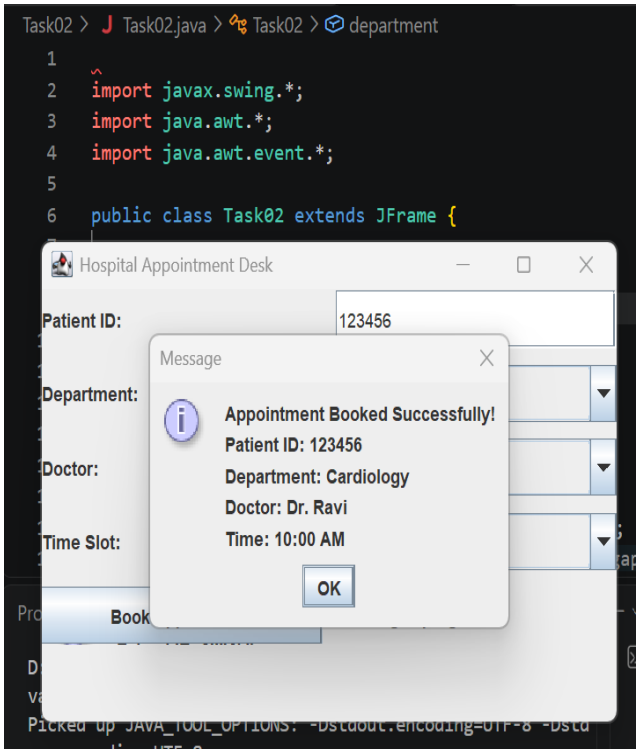
```
department = new JComboBox<>(new String[]{
    "Cardiology", "Neurology", "Orthopedics"
});
doctor = new JComboBox<>();
timeSlot = new JComboBox<>(new String[]{
    "10:00 AM", "11:00 AM", "02:00 PM", "04:00 PM"
});
book = new JButton("Book Appointment");
status = new JLabel("Enter patient details");
add(new JLabel("Patient ID:"));
add(patientId);
add(new JLabel("Department:"));
add(department);
add(new JLabel("Doctor:"));
add(doctor);
add(new JLabel("Time Slot:"));
add(timeSlot);
add(book);
add(status);
department.addActionListener(e -> updateDoctors());
book.addActionListener(e -> bookAppointment());
updateDoctors();
addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
```

```
        if (bookingInProgress) {
            int choice = JOptionPane.showConfirmDialog(
                Task02.this,
                "Booking is in progress. Do you want to exit?",
                "Confirm Exit",
                JOptionPane.YES_NO_OPTION
            );
            if (choice == JOptionPane.YES_OPTION) {
                worker.cancel(true);
                dispose();
            }
        } else {
            int choice = JOptionPane.showConfirmDialog(
                Task02.this,
                "Are you sure you want to close?",
                "Confirm Exit",
                JOptionPane.YES_NO_OPTION
            );
            if (choice == JOptionPane.YES_OPTION) {
                dispose();
            }
        }
    }
});
```

```
    setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
    setSize(450, 300);
    setLocationRelativeTo(null);
}
void updateDoctors() {
    doctor.removeAllItems();
    String selected = (String) department.getSelectedItem();
    if (selected.equals("Cardiology")) {
        doctor.addItem("Dr. Ravi");
        doctor.addItem("Dr. Kumar");
    } else if (selected.equals("Neurology")) {
        doctor.addItem("Dr. Priya");
        doctor.addItem("Dr. Anil");
    } else {
        doctor.addItem("Dr. Suresh");
        doctor.addItem("Dr. Meena");
    }
}
void bookAppointment() {
    String id = patientId.getText().trim();
    if (id.isEmpty()) {
        JOptionPane.showMessageDialog(
            this, "Please enter Patient ID"
        );
```

```
    return;
}
if (bookingInProgress) {
    return;
}
String dept = (String) department.getSelectedItem();
String doc = (String) doctor.getSelectedItem();
String time = (String) timeSlot.getSelectedItem();
bookingInProgress = true;
book.setEnabled(false);
status.setText("Booking in progress...");
worker = new SwingWorker<String, Void>() {
    protected String doInBackground() throws Exception {
        Thread.sleep(1500);
        return "Appointment Booked Successfully!\n"
            + "Patient ID: " + id
            + "\nDepartment: " + dept
            + "\nDoctor: " + doc
            + "\nTime: " + time;
    }
    protected void done() {
        bookingInProgress = false;
        book.setEnabled(true);
```

```
            if (!isDisplayable()) {
                return;
            }
            try {
                JOptionPane.showMessageDialog(
                    Task02.this, get()
                );
                status.setText("Appointment Confirmed");
            } catch (Exception ex) {
                status.setText("Booking failed");
                JOptionPane.showMessageDialog(
                    Task02.this, "Unable to complete booking"
                );
            }
        }
    };
    worker.execute();
  }
  public static void main(String[] args) {
    SwingUtilities.invokeLater(() ->
        new Task02().setVisible(true)
    );
  }
}
```



Q3- FROM SKETCH TO LAYOUT

Aim

To design and develop a Java Swing-based Campus Library Search Screen that allows students to search for books by title, author or category and view the available book details.

Theory

The Campus Library Search Screen is an event-driven GUI application developed using Java Swing.

- **JFrame:** Creates the main application window.
- **JTextField:** Accepts the search keyword.
- **JComboBox:** Allows users to select a search category.
- **JButton:** Performs the search operation.
- **JTable:** Displays book information.
- **JScrollPane:** Provides scrolling for the book table.
- **ActionListener:** Handles search button clicks.
- **KeyAdapter:** Allows searching when the Enter key is pressed.

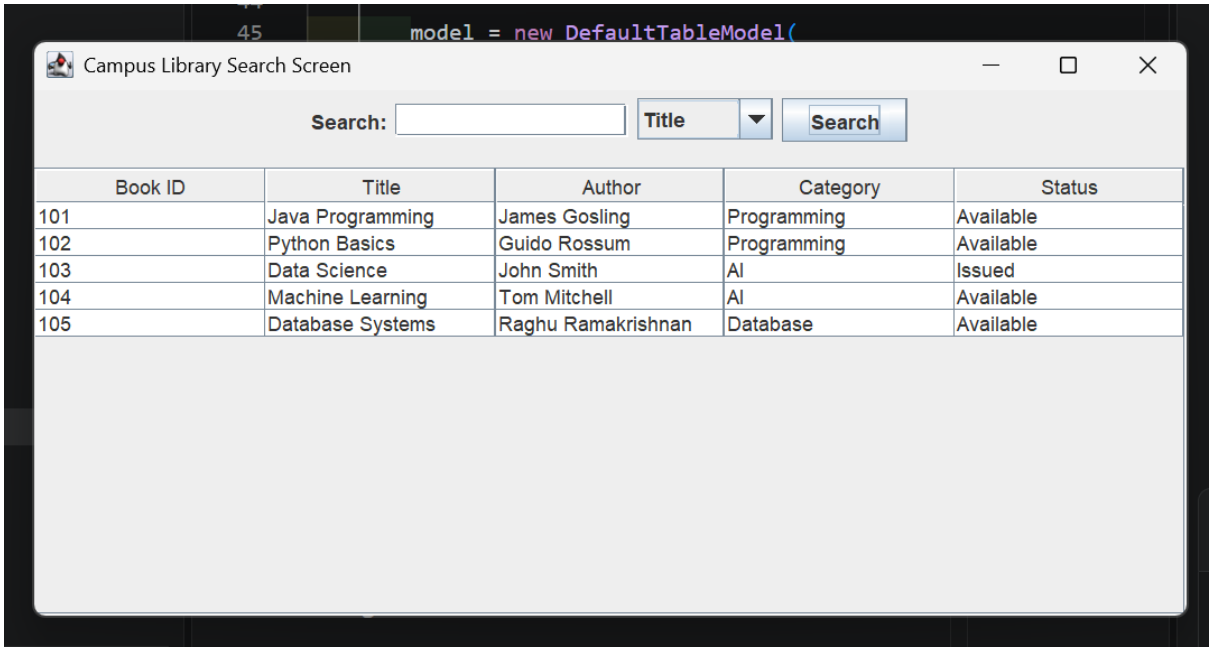
Java Program

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
public class Task03 extends JFrame {
    JTextField searchField;
    JComboBox<String> category;
    JTable table;
    DefaultTableModel model;
    JButton searchButton;
    String[][] books = {
        {"101", "Java Programming", "James Gosling", "Programming", "Available"},
        {"102", "Python Basics", "Guido Rossum", "Programming", "Available"},
        {"103", "Data Science", "John Smith", "AI", "Issued"},
        {"104", "Machine Learning", "Tom Mitchell", "AI", "Available"},
        {"105", "Database Systems", "Raghu Ramakrishnan", "Database", "Available"}
    };
    public Task03() {
        setTitle("Campus Library Search Screen");
        setLayout(new BorderLayout(10, 10));
        JPanel searchPanel = new JPanel();
        searchField = new JTextField(15);
        category = new JComboBox<>(new String[]{
```

```
    "Title", "Author", "Category"
});
searchButton = new JButton("Search");
searchPanel.add(new JLabel("Search:"));
searchPanel.add(searchField);
searchPanel.add(category);
searchPanel.add(searchButton);
add(searchPanel, BorderLayout.NORTH);
model = new DefaultTableModel(
    new String[]{"Book ID", "Title", "Author",
                "Category", "Status"}, 0
);
table = new JTable(model);
add(new JScrollPane(table), BorderLayout.CENTER);
searchButton.addActionListener(e -> searchBooks());
searchField.addKeyListener(new KeyAdapter() {
    public void keyPressed(KeyEvent e) {
        if (e.getKeyCode() == KeyEvent.VK_ENTER) {
            searchBooks();
        }
    }
});
displayBooks(books);
setSize(700, 350);
```

```
    setLocationRelativeTo(null);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
void displayBooks(String[][] results) {
    model.setRowCount(0);
    for (String[] book : results) {
        model.addRow(book);
    }
}
void searchBooks() {
    String keyword = searchField.getText()
                            .trim()
                            .toLowerCase();
    String selected = (String) category.getSelectedItem();
    if (keyword.isEmpty()) {
        JOptionPane.showMessageDialog(
            this, "Please enter a search keyword"
        );
        displayBooks(books);
        return;
    }
    model.setRowCount(0);
    for (String[] book : books) {
```

```
            String value = "";
            if (selected.equals("Title")) {
                value = book[1];
            } else if (selected.equals("Author")) {
                value = book[2];
            } else if (selected.equals("Category")) {
                value = book[3];
            }
            if (value.toLowerCase().contains(keyword)) {
                model.addRow(book);
            }
        }
        if (model.getRowCount() == 0) {
            JOptionPane.showMessageDialog(
                this, "No books found"
            );
        }
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() ->
            new Task03().setVisible(true)
        );
    }
}
```



Q4– EVENT-HANDLING FORENSICS

Aim

To analyze and debug Java Swing event-handling problems, identify incorrect listener implementations and demonstrate the correct use of ActionListener, MouseListener and adapter classes.

Theory

Event handling is the process of responding to user actions in a GUI application.

Java uses the Delegation Event Model, which consists of:

1. **Event Source:** The component that generates an event, such as a button.
2. **Event Object:** Contains information about the event.
3. **Event Listener:** Receives and handles the event.

Java Program

```
import javax.swing.*;

import java.awt.*;

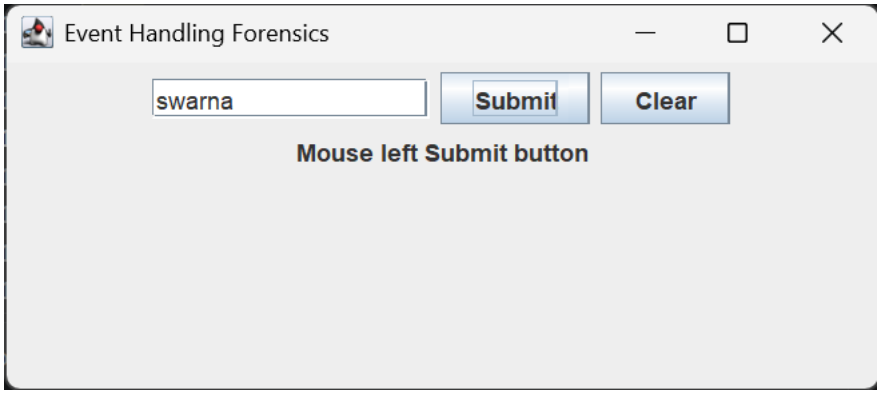
import java.awt.event.*;

public class Task04 extends JFrame {
```

```
JButton button, clearButton;
JLabel message;
JTextField input;
public Task04() {
    setTitle("Event Handling Forensics");
    setLayout(new FlowLayout());
    input = new JTextField(15);
    button = new JButton("Submit");
    clearButton = new JButton("Clear");
    message = new JLabel("Enter your name");
    add(input);
    add(button);
    add(clearButton);
    add(message);
    // ActionListener for Submit button
    button.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            String name = input.getText().trim();
            if (name.isEmpty()) {
                message.setText("Please enter your name");
            } else {
                message.setText("Welcome " + name);
            }
        }
```

```
});
// ActionListener using lambda expression
clearButton.addActionListener(e -> {
    input.setText("");
    message.setText("Fields cleared");
});
// MouseAdapter example
button.addMouseListener(new MouseAdapter() {
    public void mouseEntered(MouseEvent e) {
        message.setText("Mouse entered Submit button");
    }
    public void mouseExited(MouseEvent e) {
        message.setText("Mouse left Submit button");
    }
});
// WindowAdapter example
addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        int choice = JOptionPane.showConfirmDialog(
            Task04.this,
            "Do you want to exit?",
            "Confirm Exit",
            JOptionPane.YES_NO_OPTION
        );
```

```
            if (choice == JOptionPane.YES_OPTION) {
                dispose();
            }
        }
    });
    setSize(450, 200);
    setLocationRelativeTo(null);
    setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
  }
  public static void main(String[] args) {
    SwingUtilities.invokeLater(() ->
        new Task04().setVisible(true)
    );
  }
}
```



Q5– SAME SCREEN, DIFFERENT EVENT-HANDLING STYLES

Aim

To develop a Java Swing application using different event-handling styles, such as anonymous inner classes, lambda expressions and separate listener classes, and compare their implementation.

Theory

Java supports different ways of handling GUI events.

1. **Anonymous Inner Class:** Creates a listener directly inside the component registration.
2. **Lambda Expression:** Provides a shorter way to implement functional interfaces such as ActionListener.
3. **Separate Listener Class:** Creates a separate class to handle events.
4. **ActionListener:** Handles button-click events.

Java Program

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;

public class Task05 extends JFrame {

   JTextField number1, number2;

   JLabel result;

   public Task05() {

      setTitle("Different Event Handling Styles");

      setLayout(new FlowLayout());

      number1 = new JTextField(8);

      number2 = new JTextField(8);

      JButton anonymousButton = new JButton("Anonymous");

      JButton lambdaButton = new JButton("Lambda");

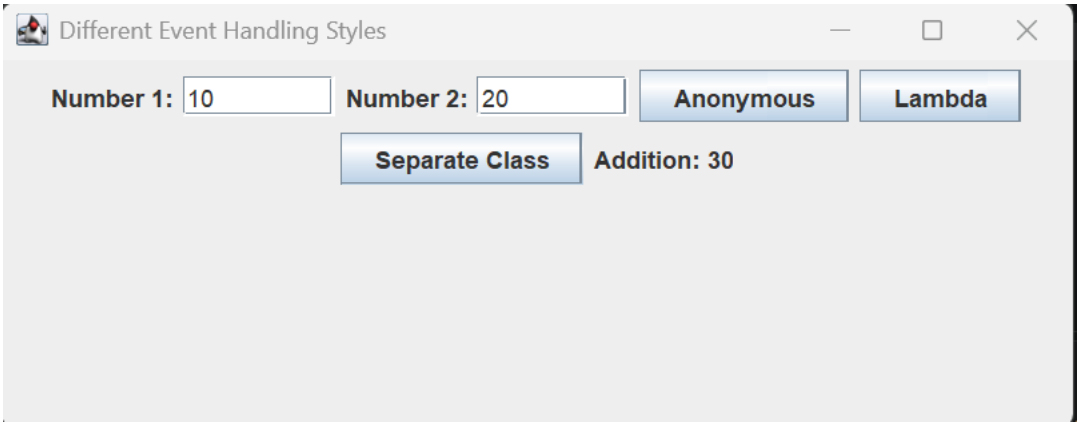
      JButton separateButton = new JButton("Separate Class");
```

```
result = new JLabel("Enter two numbers");
add(new JLabel("Number 1:"));
add(number1);
add(new JLabel("Number 2:"));
add(number2);
add(anonymousButton);
add(lambdaButton);
add(separateButton);
add(result);
// 1. Anonymous Inner Class
anonymousButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        calculate();
    }
});
// 2. Lambda Expression
lambdaButton.addActionListener(e -> calculate());
// 3. Separate Listener Class
separateButton.addActionListener(
    new SeparateListener(this)
);
setSize(550, 220);
setLocationRelativeTo(null);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
    }
    void calculate() {
        try {
            int a = Integer.parseInt(number1.getText());
            int b = Integer.parseInt(number2.getText());
            result.setText("Addition: " + (a + b));
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(
                this, "Please enter valid numbers"
            );
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() ->
            new Task05().setVisible(true)
        );
    }
}
// Separate Listener Class
class SeparateListener implements ActionListener {
    Task05 window;
    SeparateListener(Task05 window) {
        this.window = window;
```

```
    }
    public void actionPerformed(ActionEvent e) {
        window.calculate();
    }
}
```



Q6– Campus Event Registration Desktop Application

Aim

To design and develop a Campus Event Registration Desktop Application using Java Swing components and event handling. The application allows students to enter their details, select campus events, choose participation mode and register successfully.

Theory

The Campus Event Registration Application is developed using Java Swing and the Delegation Event Model.

- **JFrame:** Creates the main registration window.
- **JTextField:** Accepts student name and register number.
- **JComboBox:** Allows selection of campus events.
- **JRadioButton:** Selects participation mode.
- **JCheckBox:** Accepts registration confirmation.
- **JButton:** Performs registration and clearing operations.

- **JOptionPane:** Displays confirmation and error messages.
- **ActionListener:** Handles button clicks.
- **ItemListener:** Handles event and participation selections.
- **WindowAdapter:** Handles window-closing confirmation.

Java Program

EventService.java

```
import java.sql.*;
import java.util.ArrayList;
public class EventService {
    private final String url =
        "jdbc:mysql://localhost:3306/campus_events";
    private final String user = "root";
    private final String password = "YOUR_MYSQL_PASSWORD";
    private Connection connect() throws SQLException {
        return DriverManager.getConnection(url, user, password);
    }
    public void register(String id, String name,
                         String department, String event)
                         throws SQLException {
        String sql = "INSERT INTO registrations " +
            "(student_id, student_name, department, event_name) " +
            "VALUES (?, ?, ?, ?)";
        try (Connection con = connect();
             PreparedStatement ps = con.prepareStatement(sql)) {
```

```
        ps.setString(1, id);
        ps.setString(2, name);
        ps.setString(3, department);
        ps.setString(4, event);
        ps.executeUpdate();
    }
}
public ArrayList<String[]> getRegistrations(String event)
        throws SQLException {
    ArrayList<String[]> records = new ArrayList<>();
    String sql = "SELECT student_id, student_name, " +
        "department, event_name, registration_date " +
        "FROM registrations";
    if (!event.equals("All Events")) {
        sql += " WHERE event_name = ?";
    }
    sql += " ORDER BY registration_date DESC";
    try (Connection con = connect();
         PreparedStatement ps = con.prepareStatement(sql)) {
        if (!event.equals("All Events")) {
            ps.setString(1, event);
        }
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
```

```
                records.add(new String[]{
                    rs.getString("student_id"),
                    rs.getString("student_name"),
                    rs.getString("department"),
                    rs.getString("event_name"),
                    rs.getString("registration_date")
                });
            }
        }
        return records;
    }
}
```

CampusEventApp.java

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.util.ArrayList;
public class CampusEventApp extends JFrame {
    JTextField idField, nameField, departmentField;
    JComboBox<String> eventBox, filterBox;
    JTable table;
```

```
DefaultTableModel model;
JButton registerButton, clearButton, viewButton, exportButton;
EventService service = new EventService();
String[] events = {
    "Cultural Fest",
    "Music Competition",
    "Dance Competition",
    "Technical Symposium"
};
public CampusEventApp() {
    setTitle("Campus Event Registration");
    setLayout(new BorderLayout(10, 10));
    // Registration panel
    JPanel panel = new JPanel(new GridLayout(5, 2, 10, 10));
    idField = new JTextField();
    nameField = new JTextField();
    departmentField = new JTextField();
    eventBox = new JComboBox<>(events);
    registerButton = new JButton("Register");
    clearButton = new JButton("Clear");
    panel.add(new JLabel("Student ID:"));
    panel.add(idField);
    panel.add(new JLabel("Student Name:"));
    panel.add(nameField);
```

```
panel.add(new JLabel("Department:"));
panel.add(departmentField);
panel.add(new JLabel("Select Event:"));
panel.add(eventBox);
panel.add(registerButton);
panel.add(clearButton);
add(panel, BorderLayout.NORTH);
// Committee table
model = new DefaultTableModel(
    new String[]{"Student ID", "Name", "Department",
                "Event", "Registration Date"}, 0
);
table = new JTable(model);
add(new JScrollPane(table), BorderLayout.CENTER);
// Filter and export panel
JPanel bottom = new JPanel();
filterBox = new JComboBox<>(new String[]{
    "All Events", "Cultural Fest",
    "Music Competition", "Dance Competition",
    "Technical Symposium"
});
viewButton = new JButton("View Registrations");
exportButton = new JButton("Export CSV");
bottom.add(new JLabel("Filter Event:"));
```

```
bottom.add(filterBox);

bottom.add(viewButton);

bottom.add(exportButton);

add(bottom, BorderLayout.SOUTH);

// Menu bar

JMenuBar bar = new JMenuBar();

JMenu fileMenu = new JMenu("File");

JMenu studentMenu = new JMenu("Student");

JMenu committeeMenu = new JMenu("Committee");

JMenu helpMenu = new JMenu("Help");

JMenuItem exit = new JMenuItem("Exit");

JMenuItem clear = new JMenuItem("Clear");

JMenuItem view = new JMenuItem("View Registrations");

JMenuItem about = new JMenuItem("About");

fileMenu.add(exit);

studentMenu.add(clear);

committeeMenu.add(view);

helpMenu.add(about);

bar.add(fileMenu);

bar.add(studentMenu);

bar.add(committeeMenu);

bar.add(helpMenu);

setJMenuBar(bar);
```

```
    // Event handling
    registerButton.addActionListener(e -> registerStudent());
    clearButton.addActionListener(e -> clearFields());
    clear.addActionListener(e -> clearFields());
    viewButton.addActionListener(e -> loadRecords());
    view.addActionListener(e -> loadRecords());
    filterBox.addActionListener(e -> loadRecords());
    exportButton.addActionListener(e -> exportCSV());
    exit.addActionListener(e -> confirmExit());
    about.addActionListener(e ->
        JOptionPane.showMessageDialog(
            this, "Campus Event Registration System"
        )
    );
    addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            confirmExit();
        }
    });
    setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
    setSize(850, 500);
    setLocationRelativeTo(null);
    loadRecords();
}
```

```
void registerStudent() {
    String id = idField.getText().trim();
    String name = nameField.getText().trim();
    String department = departmentField.getText().trim();
    String event = (String) eventBox.getSelectedItem();
    if (id.isEmpty() || name.isEmpty()
            || department.isEmpty()) {
        JOptionPane.showMessageDialog(
            this, "Please fill all fields"
        );
        return;
    }
    registerButton.setEnabled(false);
    SwingWorker<Void, Void> worker =
        new SwingWorker<Void, Void>() {
        protected Void doInBackground() throws Exception {
            service.register(id, name, department, event);
            return null;
        }
        protected void done() {
            registerButton.setEnabled(true);
            try {
                get();
                JOptionPane.showMessageDialog(
```

```
                CampusEventApp.this,
                "Registration Successful!"
            );
            clearFields();
            loadRecords();
        } catch (Exception ex) {
            Throwable cause = ex.getCause();
            if (cause instanceof java.sql.SQLException
                    && ((java.sql.SQLException) cause)
                       .getSQLState().startsWith("23")) {
                JOptionPane.showMessageDialog(
                    CampusEventApp.this,
                    "Duplicate registration not allowed"
                );
            } else {
                JOptionPane.showMessageDialog(
                    CampusEventApp.this,
                    "Database error: " + ex.getMessage()
                );
            }
        }
    }
};
```

```
    worker.execute();
}
void loadRecords() {
    String selected =
        (String) filterBox.getSelectedItem();
    viewButton.setEnabled(false);
    SwingWorker<ArrayList<String[]>, Void> worker =
        new SwingWorker<ArrayList<String[]>, Void>() {
        protected ArrayList<String[]> doInBackground()
                throws Exception {
            return service.getRegistrations(selected);
        }
        protected void done() {
            viewButton.setEnabled(true);
            try {
                model.setRowCount(0);
                for (String[] row : get()) {
                    model.addRow(row);
                }
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(
                    CampusEventApp.this,
                    "Unable to load registrations: "
                    + ex.getMessage()
```

```
                );
            }
        }
    };
    worker.execute();
}
void clearFields() {
    idField.setText("");
    nameField.setText("");
    departmentField.setText("");
    eventBox.setSelectedIndex(0);
}
void exportCSV() {
    JFileChooser chooser = new JFileChooser();
    chooser.setSelectedFile(
        new File("CampusRegistrations.csv")
    );
    if (chooser.showSaveDialog(this)
            != JFileChooser.APPROVE_OPTION) {
        return;
    }
    File file = chooser.getSelectedFile();
    try (PrintWriter out = new PrintWriter(file)) {
```

```
    out.println(
        "Student ID,Name,Department,Event,Registration Date"
    );
    for (int i = 0; i < model.getRowCount(); i++) {
        for (int j = 0; j < model.getColumnCount(); j++) {
            String value =
                String.valueOf(model.getValueAt(i, j));
            value = "\"" +
                value.replace("\"", "\"\"") + "\"";
            out.print(value);
            if (j < model.getColumnCount() - 1) {
                out.print(",");
            }
        }
        out.println();
    }
    JOptionPane.showMessageDialog(
        this, "CSV exported successfully!"
    );
} catch (Exception ex) {
    JOptionPane.showMessageDialog(
        this, "Export failed: " + ex.getMessage()
    );
}
```

```
    }
    void confirmExit() {
        int choice = JOptionPane.showConfirmDialog(
            this,
            "Do you want to exit?",
            "Confirm Exit",
            JOptionPane.YES_NO_OPTION
        );
        if (choice == JOptionPane.YES_OPTION) {
            dispose();
        }
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() ->
            new CampusEventApp().setVisible(true)
        );
    }
}
```

